

 ChatGPT 

Accedi

Registrati gratuitamente

Sto creando un progetto typescript per un giochino arcade html5. Ci saranno alcuni pezzi di codice che disegneranno su un canvas nell html, mentre altri pezzi di codice che si occupano della logica pura del gioco. Come organizzare il progetto? Sii piú dettagliato possibile



Per un gioco arcade HTML5 in TypeScript, conviene separare **logica di gioco, rendering, input, asset e infrastruttura**. L'errore piú comune nei progetti piccoli è mischiare tutto dentro classi tipo `Player` che contemporaneamente aggiornano la logica, leggono la tastiera e disegnano sul canvas. All'inizio sembra semplice, ma cresce male.

Una struttura che scala bene è questa:

```
src/
|
|— core/
|   |— Game.ts
|   |— GameLoop.ts
|   |— Time.ts
|   └— EventBus.ts
|
|— game/
|   |— entities/
|       |— Player.ts
|       |— Enemy.ts
|       └— Bullet.ts
|
|   |— systems/
|       |— MovementSystem.ts
|       |— CollisionSystem.ts
|       └— SpawnSystem.ts
|
|   |— world/
|       |— GameWorld.ts
|       └— Level.ts
|
|   └— rules/
|       |— Scoring.ts
|       └— GameState.ts
|
|— rendering/
|   |— Renderer.ts
|   |— CanvasRenderer.ts
|
|   |— sprites/
|       └— SpriteManager.ts
|
|   └— renderers/
|       |— PlayerRenderer.ts
|       |— EnemyRenderer.ts
|       └— BulletRenderer.ts
|
|— input/
|   |— Keyboard.ts
|   |— Mouse.ts
|   └— InputState.ts
|
|— assets/
|   |— images/
|   └— sounds/
```

```
├── ui/
│   ├── Hud.ts
│   ├── MainMenu.ts
│   └── PauseScreen.ts
├── utils/
│   ├── Vector2.ts
│   ├── Rectangle.ts
│   ├── Random.ts
│   └── MathUtils.ts
├── types/
│   └── common.ts
└── main.ts
```

1. Separare logica e rendering

Questa è probabilmente la decisione architeturale più importante.

Sbagliato

TypeScript

```
class Player {
  update(dt: number) {
    this.x += this.speed * dt;
  }

  draw(ctx: CanvasRenderingContext2D) {
    ctx.fillRect(this.x, this.y, 50, 50);
  }
}
```

Qui il player conosce il canvas.

Meglio

```
class Player {  
  position: Vector2;  
  velocity: Vector2;  
  
  update(dt: number) {  
    this.position.x += this.velocity.x * dt;  
  }  
}
```

Rendering separato:

TypeScript

```
class PlayerRenderer {  
  render(  
    ctx: CanvasRenderingContext2D,  
    player: Player  
  ) {  
    ctx.fillRect(  
      player.position.x,  
      player.position.y,  
      50,  
      50  
    );  
  }  
}
```

Vantaggi:

- test della logica senza canvas
- possibilità di cambiare renderer
- codice più pulito

2. Il Game Loop

Centralizza tutto.

TypeScript

```
        requestAnimationFrame(this.tick);
    }

    private tick = (timestamp: number) => {
        const dt = this.calculateDelta(timestamp);

        this.update(dt);
        this.render();

        requestAnimationFrame(this.tick);
    };
}
```

Flusso tipico

```
requestAnimationFrame
    ↓
input
    ↓
update logica
    ↓
collisioni
    ↓
spawn
    ↓
render
```

3. Game World

Mantieni tutto lo stato del gioco in un unico posto.

TypeScript

```
class GameWorld {
    player: Player;
    enemies: Enemy[];
    bullets: Bullet[];
}
```

TypeScript

```
collisionSystem.update(world);  
movementSystem.update(world);  
spawnSystem.update(world);
```

4. Sistemi invece di logica distribuita

In un arcade spesso è meglio avere sistemi.

Invece di:

TypeScript

```
enemy.update();  
bullet.update();  
player.update();
```

Usa:

TypeScript

```
movementSystem.update(world);  
collisionSystem.update(world);  
lifetimeSystem.update(world);
```

Esempio:

TypeScript

```
        for (const enemy of world.enemies) {
            enemy.position.x += enemy.velocity.x * dt;
        }

        for (const bullet of world.bullets) {
            bullet.position.x += bullet.velocity.x * dt;
        }
    }
}
```

Questo approccio diventa molto potente quando il gioco cresce.

5. Input centralizzato

Evita:

TypeScript

```
window.addEventListener(...)
```

sparsi ovunque.

Crea:

TypeScript

```
class Keyboard {
    private pressed = new Set<string>();

    isPressed(key: string): boolean {
        return this.pressed.has(key);
    }
}
```

Uso:

TypeScript

```
}
```

6. Macchina a stati del gioco

Un arcade ha stati diversi.

```
MENU  
PLAYING  
PAUSED  
GAME_OVER
```

Definisci:

TypeScript

```
enum GameState {  
    Menu,  
    Playing,  
    Paused,  
    GameOver  
}
```

Poi:

TypeScript

```
switch(state) {  
    case GameState.Playing:  
        updateGame();  
        break;  
  
    case GameState.Paused:  
        break;  
}
```

7. Renderer principale

Un solo punto di ingresso.

TypeScript

```
class CanvasRenderer {  
  
    render(world: GameWorld) {  
  
        this.clear();  
  
        this.playerRenderer.render(  
            this.ctx,  
            world.player  
        );  
  
        for (const enemy of world.enemies) {  
            this.enemyRenderer.render(  
                this.ctx,  
                enemy  
            );  
        }  
    }  
}
```

8. Asset Manager

Non caricare immagini sparse nel codice.

TypeScript

```
class AssetLoader {  
  
    private images = new Map<  
        string,  
        HTMLImageElement  
    >();  
  
    async loadImage(  
        id: string,  
        url: string  
    ) {}  
}
```

TypeScript

```
const ship =  
  assets.getImage("player");
```

9. Utility geometriche

Crea oggetti riutilizzabili.

TypeScript

```
class Vector2 {  
  constructor(  
    public x = 0,  
    public y = 0  
  ) {}  
}
```

TypeScript

```
class Rectangle {  
  constructor(  
    public x: number,  
    public y: number,  
    public width: number,  
    public height: number  
  ) {}  
}
```

Collisione:

TypeScript

```
rectangleA.intersects(rectangleB);
```

10. Eventi di gioco

TypeScript

```
eventBus.emit(  
  "enemyKilled",  
  enemy  
);
```

Altrove:

TypeScript

```
eventBus.on(  
  "enemyKilled",  
  () => score += 100  
);
```

Così il sistema punteggio non conosce il sistema collisioni.

11. Testabilità

La logica dovrebbe essere indipendente dal browser.

Ad esempio:

TypeScript

```
class CollisionSystem {  
  update(world: GameWorld) {}  
}
```

può essere testata con:

TypeScript

```

    const world = createWorld();

    collisionSystem.update(world);

    expect(world.enemies.length)
      .toBe(0);
  });

```

Senza canvas.

12. Dipendenze tra moduli

Una regola molto utile:

```

game/
  NON conosce rendering/

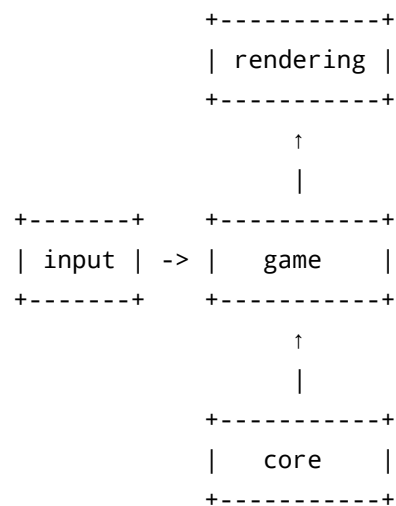
rendering/
  conosce game/

input/
  NON conosce rendering/

ui/
  può conoscere game e rendering

```

Diagramma:



La logica del gioco deve essere il cuore del progetto e non deve dipendere dal canvas.

Struttura consigliata per un arcade medio

Se dovessi iniziare oggi un clone di Space Invaders, Asteroids o un bullet hell semplice, userei:

```
src/  
├── core/  
├── game/  
│   ├── entities/  
│   ├── systems/  
│   ├── world/  
│   └── rules/  
├── rendering/  
├── input/  
├── assets/  
├── ui/  
├── utils/  
└── main.ts
```

con una regola ferrea:

Le classi dentro `game/` non devono mai importare `CanvasRenderingContext2D`, `HTMLImageElement` o qualsiasi API grafica del browser. Tutta la logica deve poter funzionare anche in un ambiente Node.js senza canvas. Questa separazione rende il codice molto più manutenibile e facile da testare.

